

Training & Pipeline Summary

Temporal Graph Network plus four baselines on 543,615 LANL authentication events. Every figure is read from a committed artifact.

Correction to previously circulated figures

Earlier versions of this document reported test PR-AUC **0.5789** as the model's performance. It is not. `_fused_scores` in the training pipeline runs the model's output through risk fusion *before* any metric is computed, so 0.5789 measures the deployed pipeline. The baselines are scored raw – `fuse_risk` appears nowhere in the baseline module – so a fused pipeline was compared against unfused detectors.

Measured raw, the model scores **0.8210** on test. Reconstructing the published figure from the same model output plus the original linear fusion yields 0.5725, within 0.0064 of 0.5789, which accounts for the difference. Both numbers are correct; they measure different things.

1. Detectors, compared like for like

The primary metric is **PR-AUC**. At 0.109% prevalence a detector answering "benign" for everything is 99.89% accurate and detects nothing, and ROC-AUC stays near 1.0 while an unusable number of false positives sit above the true ones. All figures below are **raw detector scores**.

Detector	Test PR-AUC	95% CI	Test FP/10k
TGN (model)	0.8210	[0.7639, 0.8784]	34.6
Logistic	0.2463	[0.1818, 0.3273]	59.8
Rule	0.0706	[0.0428, 0.1106]	14.3
Isolation Forest	0.0406	[0.0205, 0.0837]	29.9
Rarity	0.0072	[0.0050, 0.0124]	5.6

All five detectors were re-run to produce these intervals, and the baseline point estimates reproduce the committed report to floating-point precision (maximum deviation 2.8e-17). 2,000 stratified bootstrap resamples; the test partition holds only **126 attacks**, so the intervals are wide and no point estimate should be quoted alone.

1.1 Model against each baseline, paired

Comparison	Difference	95% CI	Significant
tgn - logistic	+0.5747	[+0.4917, +0.6483]	yes
tgn - rule	+0.7505	[+0.6887, +0.8026]	yes
tgn - isolation_forest	+0.7804	[+0.7147, +0.8353]	yes
tgn - rarity	+0.8139	[+0.7566, +0.8702]	yes

Paired on identical resampled events, so the difference does not inherit the variance of comparing two independent intervals. Every comparison is significant.

The comparison, stated correctly

Like for like, the model reaches **3.33x** the best baseline on the sealed test partition, with a conservative envelope of **2.33x–4.83x** – not the 2.35x previously stated, which divided a fused pipeline score by a raw baseline score. The direction holds and strengthens; the arithmetic was inconsistent.

The substantive result is unchanged: on **validation** logistic regression is competitive, and on the sealed **test** partition it collapses to 0.2463 while the model holds. A tabular classifier memorises which feature values marked the attacks it was shown; the temporal model learns how an entity deviates from its own history, and that transfers.

2. The promotion gate rejected the model on a handicapped comparison

The gate is *candidate beats the best declared baseline*, on validation PR-AUC. As published it compared the model's **fused** score (0.9132) against the best baseline's **raw** score (0.9163) – an improvement of -0.003023 – and recorded `eligible = false`.

Compared consistently – raw against raw – the model scores **0.9798** against the same 0.9163, an improvement of **+0.063538. The candidate would have passed.** It was rejected by 0.0030 on a comparison that cost it 0.0666.

The gate now compares like with like and records both figures, since the fused score is what the deployed pipeline produces and an operator needs both. The rejection in the committed training report stands as a fact about that run, not as a judgement about the model.

3. Data split — chronological and attack-aware

Partition	Events	Attacks	Prevalence	Role
Train	333,551	316	0.095%	fit weights
Validation	94,072	207	0.220%	early stop, threshold, promotion gate
Test	115,992	126	0.109%	sealed, scored once

543,615 events, 649 red-team, 63,397 entities. The cut is strictly chronological, so no entity state, threshold or hyperparameter choice was informed by test data.

4. Model and training

Each of 63,397 entities carries a **64-dimensional memory vector**. An event produces a 27-dimensional causal message; elapsed time since the entity was last seen is encoded through a **harmonic time encoder** into 16 dimensions, so "3 seconds later" and "3 days later" are different inputs. Message, both endpoint memories and the time encoding feed a 96-unit hidden layer that emits the risk logit; a **GRU** writes the updated memory back. Total **107,585 parameters**.

Score-before-update: what makes this leakage-safe

The model scores an event using memory as it stood *before* that event, then writes the update. Reversing those two lines would let the network see the event it is predicting, and validation PR-AUC would rise to a number that cannot survive deployment. Enforced in `TemporalGraphNetwork.step()` and asserted by a regression test.

Parameter	Value	Parameter	Value
memory_dim	64	epochs	16
time_dim	16	patience	5
hidden_dim	96	learning_rate	0.001
message_dim	27	weight_decay	0.01
dropout	0.15	positive_weight_cap	120.0
num_nodes	63,397	seed	1729
oov_user_buckets	4,096	time_bucket_seconds	60
oov_host_buckets	16,384	FP/10k budget	25.0

Trained on **CUDA**. Class imbalance uses a positive weight capped at 120x rather than the raw inverse-prevalence ratio (~1,000x), which would explode gradients on the handful of positives. **time_bucket_seconds = 60** batches events in 60-second groups instead of one batch per distinct timestamp: a measured **20.7x training and 16.9x inference speedup** with the score-before-update guarantee intact, because within a bucket memory is still read before any write.

Validation PR-AUC improved on **every one of the 16 epochs** – the best epoch is the last, so early stopping never triggered and the model had not plateaued when the budget ran out.

5. Risk scoring at serve time

The model probability is one of four channels combined by an auditable fusion operator. Three operators are calibrated, each carrying its own decision threshold – changing one without the other silently changes the alert rate:

Operator	Threshold	Test PR-AUC	95% CI	Note
linear	0.3613	0.5753	[0.4952, 0.6551]	original; gives the model an effective veto
noisy_or	0.3242	0.8041	[0.7397, 0.8664]	default; recovers most of what linear discarded
tgn_only	0.2968	0.8210	[0.7639, 0.8784]	model passthrough; highest ranking quality

Fusion is retained although the raw model ranks best, because it provides per-channel attribution an analyst can challenge, graceful degradation when no checkpoint is loaded, and a place for deterministic rules to raise a floor. That is a trade, and presenting fusion as an accuracy improvement would be false.

6. Reproducing every figure here

```
python -m graphsentinel.cli backfill --input data/processed/features_lanl_545k_split --output
artifacts/state --checkpoint artifacts/models/<checkpoint>.pt
python -m graphsentinel.cli evaluation-report --cache <scored>.parquet --output docs
python scripts/build_summary_pdf.py
```

Seed 1729 throughout. The evaluation report fixes its bootstrap seed, so two runs agree exactly and a changed number appears as a diff rather than a surprise.